tmux

front

Sessions

`tmux new -s name`	New session
`tmux ls`	List sessions
`tmux attach -t n`	Attach
`tmux kill-session -t n`	Kill
`C-a d`	Detach
`C-a s`	Session picker
`C-a $`	Rename session

Windows

`C-a c`	New window
`C-a ,`	Rename window
`C-a &`	Kill window
`C-a p / n`	Prev / next window
`C-a 1-9`	Jump to window N
`C-a w`	Window picker

Panes

`C-a |`	Split vertical
`C-a -`	Split horizontal
`C-a x`	Kill pane
`C-h/j/k/l`	Navigate (smart-splits)
`M-h/j/k/l`	Resize (smart-splits)
`C-a z`	Zoom / unzoom
`C-a { / }`	Swap pane left / right
`C-a q`	Show pane numbers

Copy Mode (vi)

`C-a [`	Enter copy mode
`v`	Begin selection
`y`	Yank to system clipboard
`q / Esc`	Exit copy mode
`/ ?`	Search forward / backward

Prefix: `C-a` vi mode mouse on base index 1

nvim: core editing

front

Motion

`h j k l`	Left, down, up, right
`w / b`	Next / prev word start
`e / ge`	Next / prev word end
`0 / $`	Line start / end
`^`	First non-blank
`gg / G`	File start / end
`{ / }`	Paragraph up / down
`%`	Matching bracket
`fc / Fc`	Find char fwd / back
`C-d / C-u`	Half-page down / up

Editing

`i / a`	Insert before / after
`I / A`	Insert at line start / end
`o / O`	New line below / above
`x`	Delete char
`dd / cc`	Delete / change line
`dw / cw`	Delete / change word
`D / C`	Delete / change to EOL
`rc`	Replace char
`~`	Toggle case
`.`	Repeat last change
`u / C-r`	Undo / redo

Visual Mode

`v / V / C-v`	Char / line / block
`< / >`	Indent (stays in visual)
`J / K`	Move selection down / up
`SPC p`	Paste without yanking

Text Objects (`d/c/y` + object)

`iw/aw` word	`i"/a"` double-quoted
`ip/ap` paragraph	`i(/a(` parentheses
`it/at` HTML tag	`i{/a{` braces

mini.surround

`samotionchar`	Add surround
`sdchar`	Delete surround
`sroldnew`	Replace surround

Example: `saiw"` surrounds word with `"`

Leader: `Space` relative lines scrolloff 8 system clipboard

nvim: core editing

back

Registers & Macros

`"ay / "ap`	Yank / paste register `a`
`"+y / "+p`	System clipboard
`:reg`	Show all registers
`qa`	Record macro into `a`
`q`	Stop recording
`@a`	Play macro `a`
`@@`	Repeat last macro
`10@a`	Play macro 10 times

Marks

`ma`	Set mark `a`
`` `a ``	Jump to exact position
`'a`	Jump to line
`` `. ``	Last edit position
`` `` ``	Previous jump position

Command Mode

`:s/old/new/g`	Substitute in line
`:%s/old/new/gc`	Substitute in file
`:g/pattern/d`	Delete matching lines
`:r !cmd`	Insert command output
`:!cmd`	Run shell command
`:norm @a`	Macro on visual sel.

Splits & Buffers

`SPC |`	Vertical split
`SPC -`	Horizontal split
`C-w =`	Equalize splits
`C-w q`	Close split
`C-h/j/k/l`	Navigate (smart-splits)
`A-h/j/k/l`	Resize (smart-splits)
`S-h / S-l`	Prev / next buffer
`SPC bd`	Delete buffer

Save & Quit

`SPC w`	Save
`SPC q`	Quit
`SPC Q`	Force quit all

File Explorer (netrw)

`SPC e / E`	Open explorer
`<CR>`	Open file / dir
`-`	Go up
`%`	New file
`d`	New directory
`D`	Delete
`R`	Rename

tmux

back

Plugins

`C-a C-s / C-r`	Save / restore sessions
`o` (copy mode)	Open selection (open)
`S` (copy mode)	DuckDuckGo search (open)
`C-a P`	Toggle pane logging
`C-a M-p`	Screen capture to file

continuum: auto-saves/restores on tmux start

Command Mode

`C-a :`	Enter command mode
`:resize-pane -D 10`	Resize down 10 rows
`:swap-window -t 3`	Move window to pos 3
`:move-pane -t sess:`	Move pane to session
`:set synchronize-panes`	Type in all panes
`:capture-pane -S -100`	Capture last 100 lines

Session Workflow

`C-a ( / )`	Prev / next session
`C-a L`	Last session (toggle)

Troubleshooting

`tmux source ~/.tmux.conf`	Reload config
`C-a I`	Install TPM plugins
`tmux kill-server`	Nuclear reset
`tmux list-keys | grep k`	Find binding

pain-control Splits

`C-a |`	Vertical split
`C-a -`	Horizontal split
`C-a < >`	Resize pane

nvim: lsp, search & git

front

LSP

`gd`	Go to definition
`gD`	Go to declaration
`grr / gri`	References / implementation
`grn / gra`	Rename / code action
`K`	Hover docs
`SPC rn`	Rename symbol
`SPC ca`	Code action
`SPC d`	Diagnostics float
`[d / ]d`	Prev / next diagnostic
`SPC th`	Toggle inlay hints
`SPC f`	Format buffer

Completion (blink.cmp)

`C-Space`	Trigger completion
`C-n / C-p`	Next / prev item
`C-y`	Accept
`C-e`	Cancel
`C-f / C-b`	Scroll docs

Telescope Search

`SPC sf`	Find files
`SPC sg`	Live grep
`SPC sw`	Grep current word
`SPC sb`	Buffers
`SPC s.`	Recent files
`SPC s?`	Help tags
`SPC sk`	Keymaps
`SPC sd`	Diagnostics
`SPC sr`	Resume last search
`SPC sn`	Nvim config files
`SPC /`	Fuzzy find in buffer

Inside: `C-n/p` navigate, `CR` select, `Esc` close, `C-x` split, `C-v` vsplit

Git via Telescope

`SPC gc`	Git commits
`SPC gf`	Git file history
`SPC gs`	Git status

LSP: bashls, pyright, lua_ls, yamlls, ts_ls, terraformls, dockerls, taplo, julials, jdtls

cli tools

front

bat (not aliased to cat)

`bat file.py`	Syntax + line numbers
`bat -l json`	Force language
`bat -p file`	Plain output
`bat -A file`	Show non-printable
`bat -diff file`	Git diff inline
`bat -r 10:20 file`	Lines 10–20 only

Also: `MANPAGER` (man pages with highlighting)

eza (ls stays stock)

`ll`	`eza -la -git -icons`
`lt`	`eza -tree -level=2 -icons`
`eza -la -sort=modified`	Sort by time
`eza -la -sort=size`	Sort by size
`eza -git-ignore`	Respect .gitignore

fzf (fuzzy finder)

`C-t`	File picker (insert)
`C-r`	History search
`M-c`	cd into directory
`vim $(fzf)`	Open file in nvim
`rg pat | fzf`	Filter grep results

Backend: `rg` for files, `fd` for dirs

fd (find replacement)

`fd pattern`	Find files
`fd -e py`	By extension
`fd -t d / -t f`	Dirs / files only
`fd -H`	Include hidden
`fd -I`	Ignore .gitignore
`fd pat -exec cmd {}`	Exec on each
`fd -e log -changed-within 1d`	Recent

ripgrep / **rg** (grep replacement)

`rg pattern`	Search recursively
`rg pattern -t py`	Python files only
`rg pattern -g '*.yaml'`	Glob filter
`rg -i pattern`	Case insensitive
`rg -w pattern`	Whole word
`rg -l pattern`	File names only
`rg -c pattern`	Count per file
`rg -C 3 pattern`	3 lines context
`rg -no-ignore pat`	Include gitignored
`rg -F 'literal'`	Fixed string

cli tools

back

JQ (JSON processor)

`jq '.'`	Pretty-print
`jq '.key'`	Extract key
`jq '.items[0]'`	First element
`jq '.items[] | .name'`	Map over array
`jq 'select(.age > 30)'`	Filter
`jq -r '.name'`	Raw (no quotes)
`jq -c '.'`	Compact output
`jq 'keys'`	Get all keys
`jq 'length'`	Count items

XH (curl alternative)

`xh url`	GET (default)
`xh POST url k=v`	POST JSON
`xh POST url k:=123`	Non-string value
`xh -f POST url k=v`	Form POST
`xh url Header:val`	Custom header
`xh -d url`	Download file
`xh -offline POST url`	Preview request

ZOXIDE (cd replacement)

`z dotfiles`	Jump to best match
`z dot`	Partial match works
`zi`	Interactive (fzf)
`z -`	Previous directory

DIRENV (per-project env)

`direnv allow`	Trust .envrc
`direnv deny`	Revoke trust

Create `.envrc` in project root. Auto-loads on `cd`.

TLDR (simplified man pages)

`tldr tar`	Quick examples
`tldr -update`	Update cache

MAKE (dotfiles)

`make`	Stow all configs
`make homebrew`	Install core packages
`make tools`	npm + uv tool installs
`make ssh`	Deploy SSH config

ALIAS TABLE

`ll`	→ `eza -la -git -icons`
`lt`	→ `eza -tree -level=2`
`vi / vim`	→ `nvim`
`man`	→ via `bat`

nvim: lsp, search & git

back

GITSIGNS (HUNKS)

`]h / [h`	Next / prev hunk
`SPC hs`	Stage hunk
`SPC hr`	Reset hunk
`SPC hb`	Blame line
`SPC hd`	Diff against index
`SPC hD`	Diff against last commit

LINT (NVIM-LINT)

`SPC ll`	Lint current file

Auto-lints on save. Linters: ruff (python), tflint (terraform).

DEBUG (NVIM-DAP)

`SPC Db / DB`	Toggle / conditional breakpoint
`SPC Dc`	Continue or start
`SPC Do / Di / DO`	Step over / into / out
`SPC Dr / Dt`	REPL / terminate
`SPC Du`	Toggle DAP UI

debugpy, lldb-dap, pwa-node, java (jdtls).

TOGGLES

`SPC tz`	Zen mode
`SPC tt`	Terminal (split)

WHICH-KEY

Press `SPC` and wait to see all bindings by group.

PLUGINS (VIM.PACK)

`:lua vim.pack.update()`	Update; `:w` to apply
`:lua vim.pack.del{..}`	Remove from disk
`:checkhealth vim.pack`	Plugin health
`:checkhealth`	Nvim diagnostics

TREESITTER

Installed: bash, c, cpp, css, diff, dockerfile, hcl, html, java, javascript, json, julia, lua, markdown, python, sql, terraform, toml, tsx, typescript, vim, yaml. Context headers (3 lines max) show enclosing function/class.

FORMATTING (CONFORM.NVIM)

Formats on save. lua: stylua, python: ruff, js/ts/json/yaml/md: prettier, terraform: terraform_fmt, toml: taplo. `SPC f` to format manually.

git

front

Your Aliases

`st`	status	`br`	branch
`sts`	status –short	`ci`	commit
`co`	checkout	`prev`	last branch
`hist`	log graph	`unstage`	restore –staged
`amend`	amend commit	`undo`	soft reset HEAD~1

Basics

`git init`	New repo
`git clone url`	Clone
`git add file`	Stage file
`git add -p`	Stage by hunk
`git ci -m "msg"`	Commit
`git push -u origin br`	Push new branch
`git pull`	Pull + rebase

Branching

`git br`	List branches
`git co -b feat`	New branch
`git switch main`	Switch to main
`git prev`	Toggle last branch
`git br -d br`	Delete merged
`git br -D br`	Force delete

Inspecting

`git st`	Status
`git diff`	Unstaged (delta)
`git diff -staged`	Staged changes
`git hist`	Pretty log graph
`git log -oneline -20`	Last 20 commits
`git show abc123`	Show a commit
`git blame file`	Line authorship
`git log -p - file`	File history

Stashing

`git stash`	Stash changes
`git stash pop`	Restore + remove
`git stash list`	List stashes
`git stash show -p`	Show contents
`git stash drop`	Delete top stash

Pager: delta (side-by-side) Pull: rebase Conflicts: zdiff3

tui tools

front

lazygit (`SPC gg` from nvim)

Panels: 1 Status 2 Files 3 Branches 4 Commits 5 Stash

`Space`	Stage / unstage
`a`	Stage all
`c / C`	Commit / with editor
`P / p`	Push / pull
`Enter`	Expand / view diff
`d`	Discard (file panel)
`s`	Squash (commits)
`r`	Reword commit
`R`	Rebase
`M`	Merge
`+ / -`	More / less context
`/`	Filter
`?`	Help
`q`	Quit

yazi (file manager, vim-like)

`yazi` to launch `yazi` *path* to open at path

`h / l`	Parent / enter dir
`j / k`	Down / up
`Enter`	Open file
`Space`	Select item
`/`	Search
`n / N`	Next / prev match
`o`	Open with default app
`y / p`	Yank / paste
`d / x`	Delete / cut
`a`	Create file
`r`	Rename
`.`	Toggle hidden
`z`	Jump (zoxide)
`~`	Home
`Tab`	Switch pane
`q`	Quit

tui tools

back

BOTTOM / BTM (system monitor)

`e`	Expand widget
`h/j/k/l`	Navigate
`/`	Process search
`Tab`	Cycle widgets
`dd`	Kill process
`c / m / p`	Sort: CPU / mem / PID
`t`	Tree view
`?`	Help
`q`	Quit

Flags: `-battery` `-b` basic mode

GLOW (markdown renderer)

`glow file.md`	Render file
`glow`	Browse markdowns
`glow -p file.md`	Pager mode
`glow -w 80 file.md`	Set width

LAZYDOCKER (Docker TUI)

`e`	Exec shell into container
`d`	Remove container
`s`	Stop container
`r`	Restart container
`a`	Attach to container
`[`	Previous tab
`]`	Next tab
`/`	Filter
`q`	Quit

SHELLCHECK (shell linter)

`shellcheck script.sh`	Lint script
`shellcheck -e SC2086`	Exclude rule
`shellcheck -f diff`	Auto-fixable diff
`shellcheck -s bash`	Force dialect

Common: SC2086 unquoted var, SC2046 unquoted sub, SC2155 declare+assign

DELTA (diff pager, auto-configured)

Your git uses delta with side-by-side + line numbers.

`delta f1 f2`	Diff two files
`n / N`	Next / prev file
`/`	Search
`q`	Quit

git

back

REBASE

`git rebase main`	Rebase onto main
`git rebase -continue`	After resolving
`git rebase -abort`	Cancel rebase

Auto-stash enabled: dirty worktree OK.

UNDOING

`git unstage file`	Unstage (alias)
`git undo`	Undo commit (alias)
`git amend`	Amend (alias)
`git checkout - file`	Discard changes
`git revert abc`	Revert commit
`git reset -soft HEAD~3`	Undo 3, keep staged
`git reset -mixed HEAD~1`	Undo + unstage

MERGE CONFLICTS (ZDIFF3)

Shows original text between both versions:

`<<<< HEAD`	Your changes
`|||||||`	Original (zdiff3)
`=======`	Their changes
`>>>> branch`	End marker

GITHUB CLI (GH)

`gh pr create`	Create PR
`gh pr view`	View current PR
`gh pr list`	List open PRs
`gh pr checkout 123`	Check out PR
`gh issue list`	List issues
`gh issue create`	Create issue
`gh repo clone u/r`	Clone repo
`gh browse`	Open in browser

CHERRY-PICK & BISECT

`git cherry-pick abc`	Apply commit here
`git bisect start`	Binary search for bug
`git bisect bad`	Mark current bad
`git bisect good abc`	Mark commit good
`git bisect reset`	Done bisecting

USEFUL PATTERNS

`git log -all -oneline -graph`	Topology
`git diff main...HEAD`	Branch diff
`git log main..HEAD`	Branch commits
`git reflog`	Lost commits

Line Editing

`C-a / C-e`	Line start / end
`M-b / M-f`	Word back / forward
`C-w`	Delete word backward
`M-d`	Delete word forward
`C-u`	Delete to line start
`C-k`	Delete to line end
`C-y`	Paste deleted text
`C-r`	History search (fzf)
`C-t`	File picker (fzf)
`M-c`	Fuzzy cd (fzf + fd)
`Tab`	Complete (menu)
`→`	Accept suggestion
`M-→`	Accept next word

History (8192, shared sessions)

`!!`	Repeat last command
`!$`	Last argument
`!abc`	Last cmd starting `abc`
`!abc:p`	Print don't execute
`history | rg pat`	Search history

Job Control

`C-z`	Suspend foreground
`bg`	Resume in background
`fg`	Resume in foreground
`jobs`	List background jobs
`kill %1`	Kill job 1

Redirection & Piping

`cmd &`	Run in background
`c1 && c2`	Run c2 if c1 succeeds
`c1 || c2`	Run c2 if c1 fails
`c1 | c2`	Pipe stdout
`c1 |& c2`	Pipe stdout+stderr
`cmd > file`	Redirect stdout
`cmd 2>&1`	Stderr to stdout
`cmd &> file`	Redirect all
`cmd » file`	Append

Shell: zsh Prompt: starship Plugins: autosuggestions, syntax-highlighting

Zsh Globbing

`**/*.py`	Recursive glob
`*.txt~R*`	Exclude pattern
`*(.)`	Regular files only
`*(/)`	Directories only
`*(om[1,5])`	5 most recent
`*(Lk+100)`	Files > 100KB
`*(@)`	Symlinks only

Parameter Expansion

`${v:-default}`	Default if unset
`${v:=default}`	Assign if unset
`${v#pat}`	Remove shortest prefix
`${v##pat}`	Remove longest prefix
`${v%pat}`	Remove shortest suffix
`${v%%pat}`	Remove longest suffix
`${v/old/new}`	Replace first
`${v//old/new}`	Replace all
`${v:u} / ${v:l}`	Upper / lower
`${#v}`	Length

Builtins

`type cmd`	What is this?
`which cmd`	Path to command
`where cmd`	All locations
`whence -v cmd`	Verbose type
`command cmd`	Bypass aliases
`source file`	Run in current shell
`exec cmd`	Replace shell

Environment

`env`	Show all vars
`export K=v`	Set env var
`unset K`	Remove var
`printenv K`	Print one var

Quick Reference

`ll` → `eza -la -git -icons`
`lt` → `eza -tree -level=2`
`vi/vim` → `nvim`
`man` → via `bat`
`cd` → `z` (zoxide)
`grep` → `rg`
`find` → `fd`

Right Option remapped to Control via hidutil